UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

IA PARA ENGENHEIROS
UFRGS

Curso de Especialização

Inteligência Artificial para Engenheiros

# *Análise de Dados e Engenharia de Características*

## Aula 08:

*Seleção de Características*

Professor: **Vinicius Horn Cene**

2026

# Direitos Autorais

Este material possui Direitos Autorais e Direitos de Imagem e não pode ser duplicado, reproduzido, no todo ou em parte, sob quaisquer formas ou por quaisquer meios (eletrônicos, mecânico, gravação, fotocópia, distribuição na internet ou outros), sem permissão expressa do autor.

©

# Termo de Responsabilidade da Própria UFRGS

Todos os materiais disponibilizados nos Ambientes Virtuais da UFRGS são exclusivamente para usos e fins didáticos, sendo vedada a sua utilização para qualquer outra finalidade, sob as penas legais.

Todos os materiais de terceiros que venham a ser utilizados ou disponibilizados devem ser referenciados, sob as penas legais, inclusive de plágio.

Tenho liberdade de expor ou não a minha imagem e voz nas atividades, o que não me desonera da participação efetiva nas mesmas.

Todas as gravações de atividades síncronas devem ser previamente informadas por parte dos professores.

Os alunos somente poderão gravar, armazenar ou compartilhar atividades mediante concordância prévia do professor, sob as penas legais.

Os alunos somente poderão armazenar ou compartilhar materiais mediante concordância prévia do professor ou se permitido no material, sob as penas legais.

É proibido disponibilizar, por quaisquer meios digitais ou físicos, os dados, a imagem e a voz dos alunos, dos colegas e do professor, sem autorização específica para a finalidade pretendida.

# Motivação

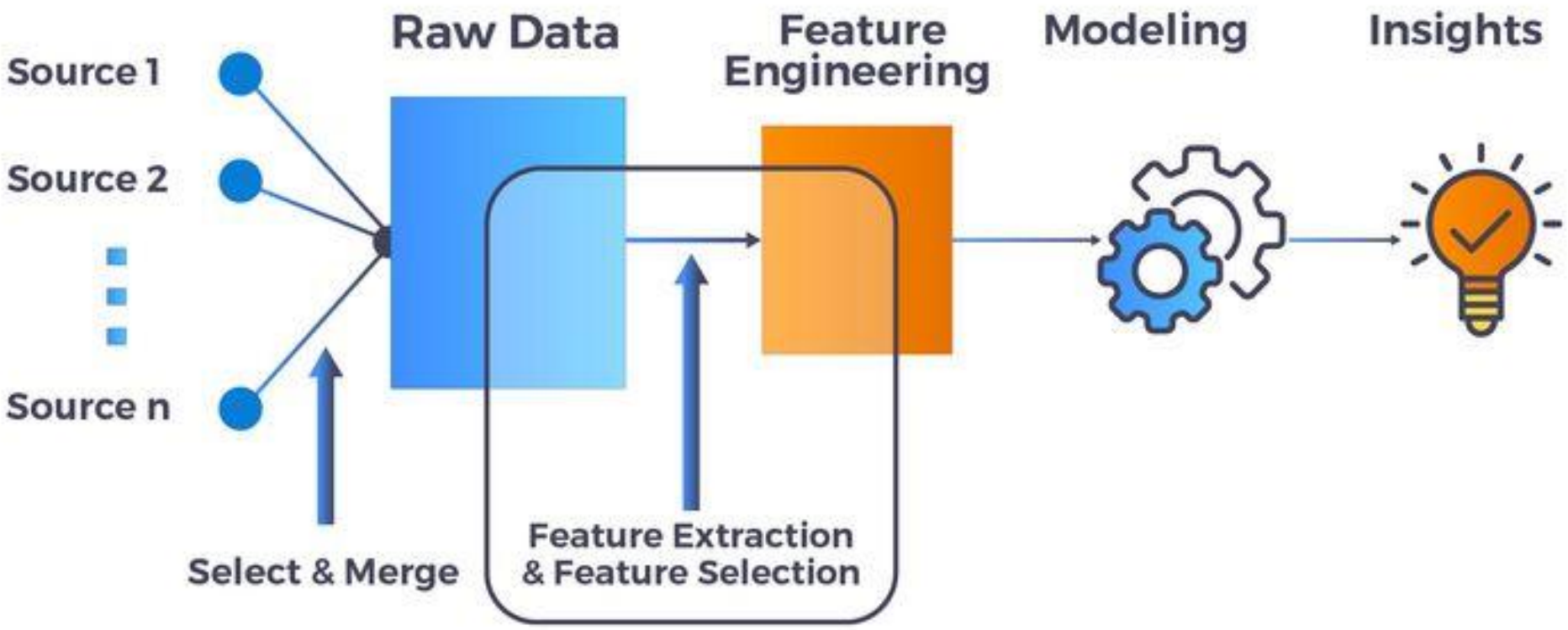
**Na última aula:**

- Transformação de variáveis categóricas em informação numérica quantitativa para os modelos
- Geração de features auxiliares para aumentar a quantidade a qualidade de informação disponível aos modelos
- Critérios de seleção para as features e os encoders de acordo com o problema e os modelos

**Objetivos da aula (ao final da aula vocês deverão):**

- **Ideia principal:** mais features não significam mais informação!
- **Métodos: Filtros, Wrappers, Embedded**
- **Efeitos** esperados da seleção e não-seleção de características
- **Estratégia** para seleção de características
- **Comparativos**, experimentos e implementação

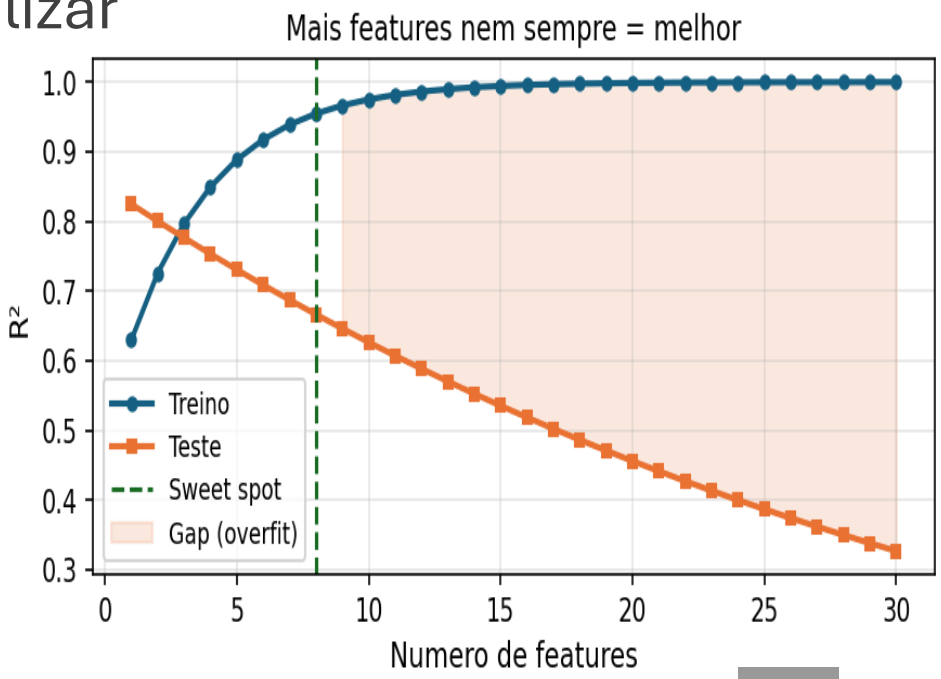
# Feature Engineering




**Fonte Imagem:** ML pipeline com ênfase em feature engineering

# Por que Selecionar Características?

- **Nem toda feature agrega sinal:**
  - features irrelevantes introduzem ruído e features redundantes inflam a dimensionalidade
- Mais features: **mais dados necessários** para generalizar
- Custo computacional cresce com colunas
- **Seleção reduz custo** de coleta, armazenamento, manutenção em produção **e** reduz **colinearidade**
- Modelos menores são mais fáceis **de auditar, explicar e atualizar**
- **Objetivo:** encontrar o subconjunto mínimo que maximize a capacidade preditiva



# A Maldição da Dimensionalidade

- Com ***p* features**, o espaço de busca cresce exponencialmente:
  - **2^p subconjuntos possíveis**
- Em **altas dimensões**, os **pontos** ficam progressivamente **mais distantes** uns dos outros
- **Distâncias perdem significado:**
  - Algoritmos baseados em vizinhança (KNN, SVM-RBF) **degradam**
- Para manter **densidade** constante, o **volume** de **dados** necessário **cresce exponencialmente**
- **Exemplo:**
  - Com 20 features binárias, seriam precisos ~1 milhão de exemplos para cobrir o espaço
- **Reduzir dimensionalidade é uma forma de combater esse efeito antes de modelar**

*CONSULTAR: Exercício 01 do script de apoio*

# Os Três Pilares da Seleção de Características

- ① **Redução de Overfitting:**
  - menos features = menos graus de liberdade = modelo mais generalizável
- ② **Interpretabilidade:**
  - Clientes entendem modelos com 10 features, não com 200
- ③ **Velocidade de Treinamento:** tempo de treino pode cair em ordens de magnitude
  - **Bônus:** reduz custo de inferência em produção (menos dados a coletar por predição)
  - **Bônus:** facilita detecção de data drift (poucos sensores são mais fáceis de monitorar)

- **Na prática, os três pilares frequentemente se reforçam mutuamente**

*CONSULTAR: Exercício 02 do script de apoio*

# Mapa Geral dos Métodos de Seleção

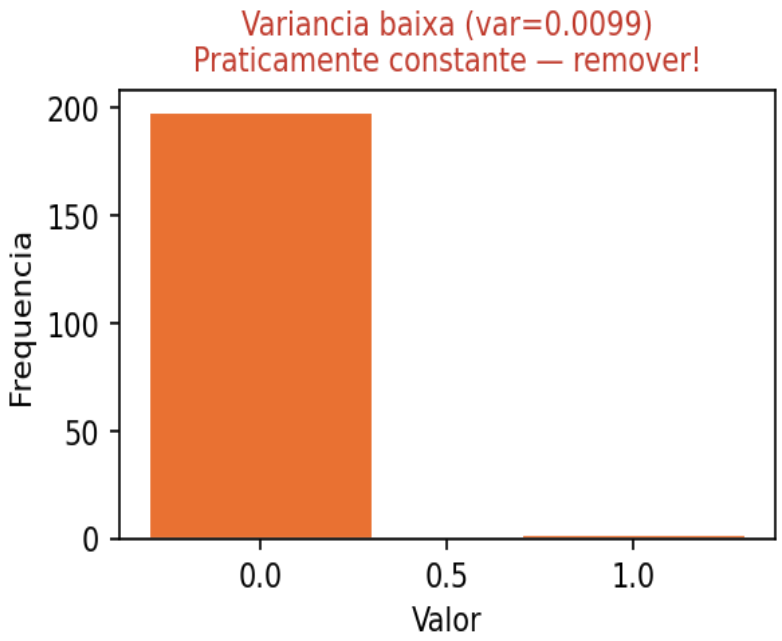
- **Filter:**
  - Avaliam features individualmente por estatísticas (correlação, chi$^2$, ANOVA)
  - Rápidos, independentes do modelo, mas ignoram interações entre features
- **Wrapper:**
  - Usam o modelo como função de avaliação de subconjuntos
  - Mais precisos, porém computacionalmente caros (treinam muitos modelos)
- **Embedded:**
  - Seleção ocorre durante o próprio treinamento do modelo
  - Equilíbrio entre custo e qualidade; Lasso, Ridge e árvores são exemplos clássicos

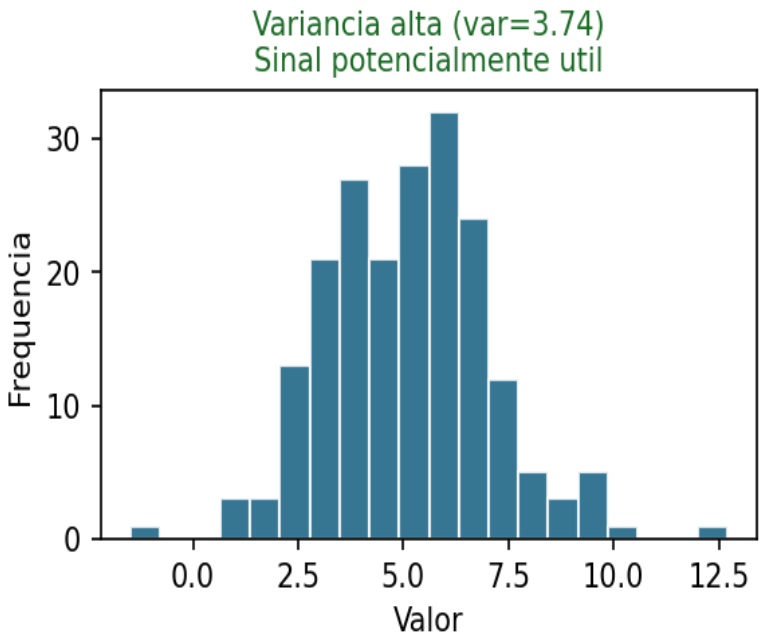
- **Esta aula apresentará algumas abordagens para as três alternativas, com foco em técnicas populares utilizadas em produção!**

*CONSULTAR: Exercício 03 do script de apoio*

# Métodos Filter

- **Ideia central:** Descartar features que não é agregam informação ou são redundantes analisando somente as características
- **Exemplo: Remoção por Variância Baixa**



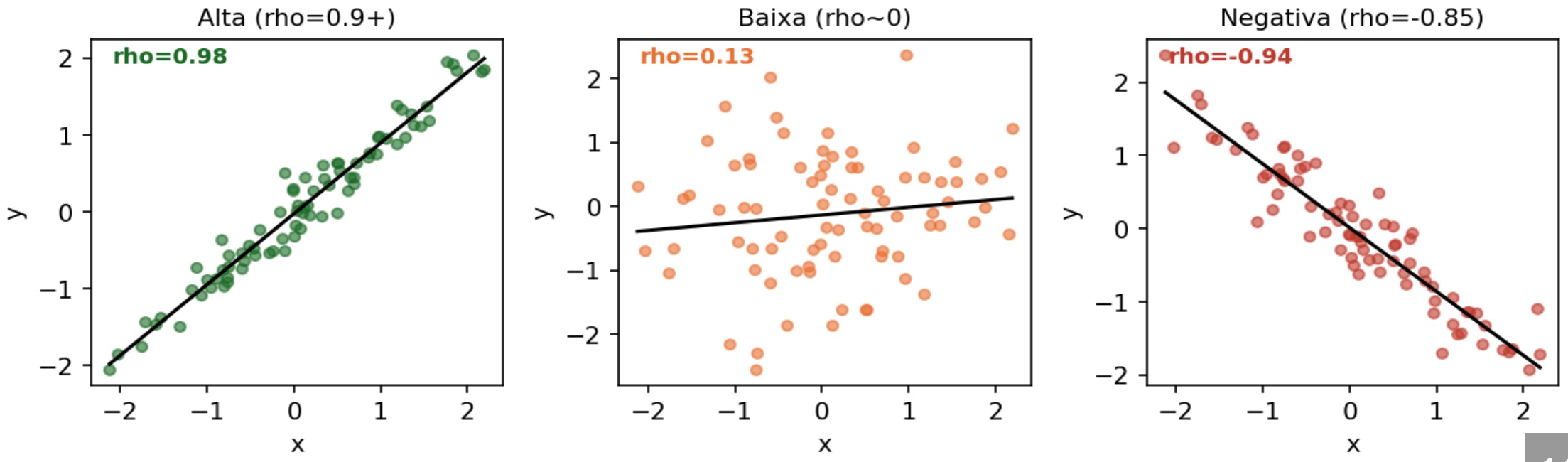




- **Features com variância ~0** são **constantes** (ou quase)!
- Não carregam informação suficiente para diferenciar as amostras
- **Threshold típico:**
  - *var < 0.01* para features binárias
- **Sklearn:**
  - *VarianceThreshold(threshold=0.01)*

# Métodos Filter

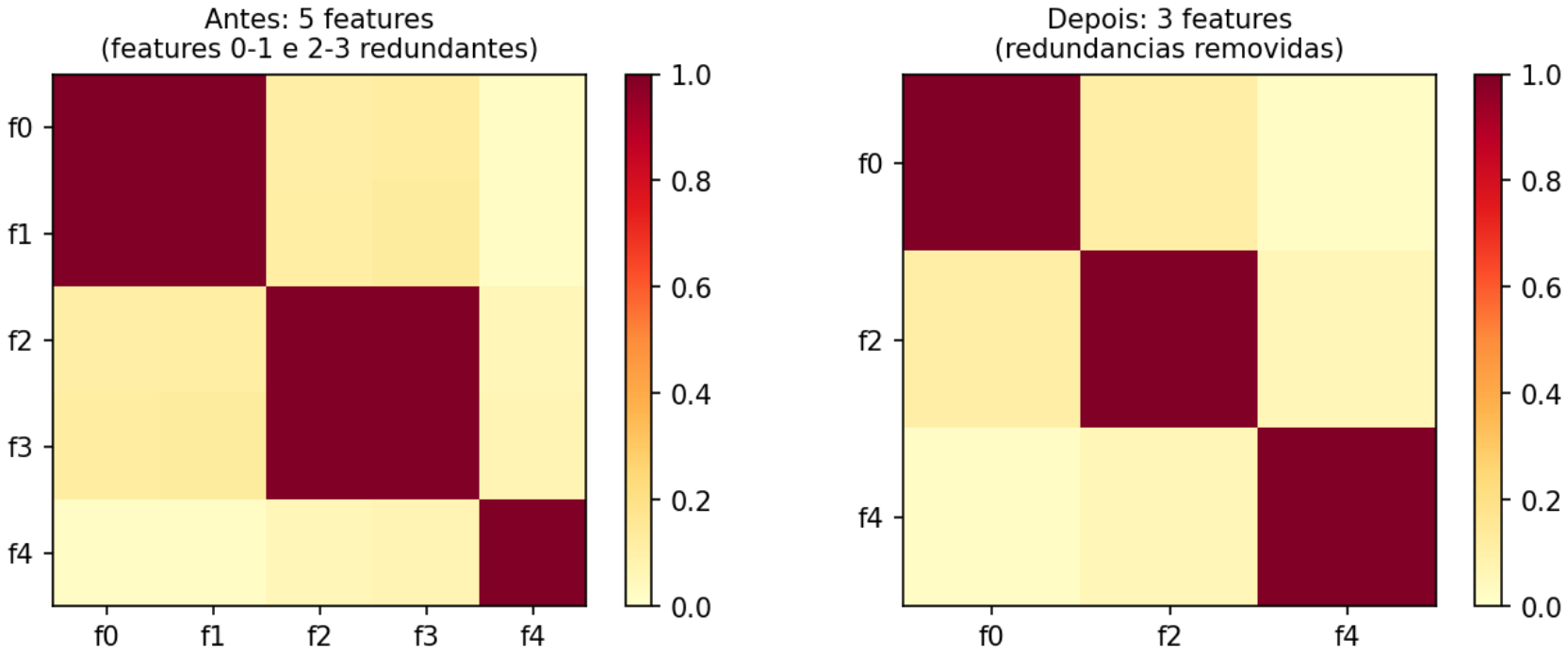
- **Ideia central:** Descartar features que não é agregam informação ou são redundantes analisando somente as características
- **Exemplo: Análise de Correlações**

**Limitação:** não detecta relações curvas, cíclicas ou de interação



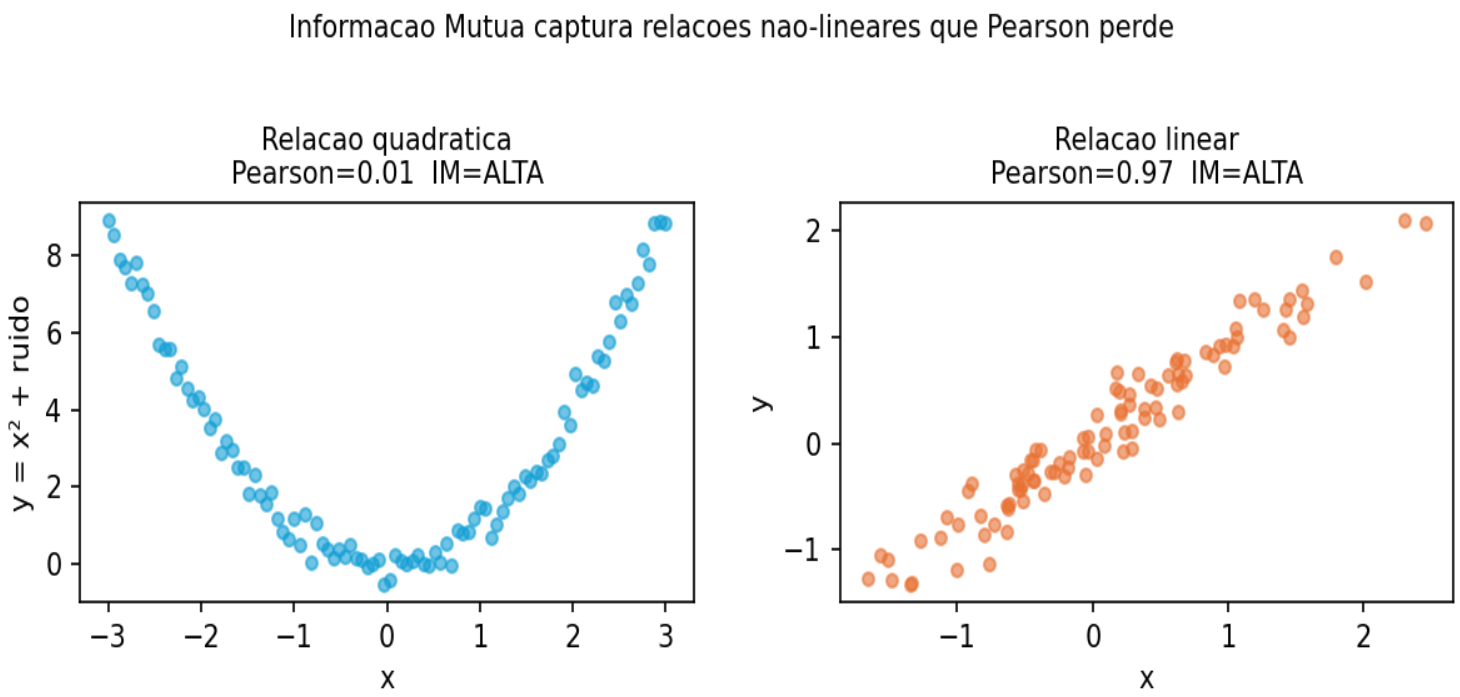
# Métodos Filter

- **Features altamente correlacionadas** entre si (|rho| > 0.9) são **redundantes**
- **Manter** as duas **não acrescenta informação**, apenas cria multicolinearidade
- **Estratégia: calcular** matriz de correlação, **identificar** pares, **manter** uma feature
- **Ordem de preferência:** manter a que tem **maior correlação com o target**



# Métodos Filter

- **Ideia central:** Descartar features que não é agregam informação ou são redundantes analisando somente as características
- **Exemplo: Análise de Informação Mútua**



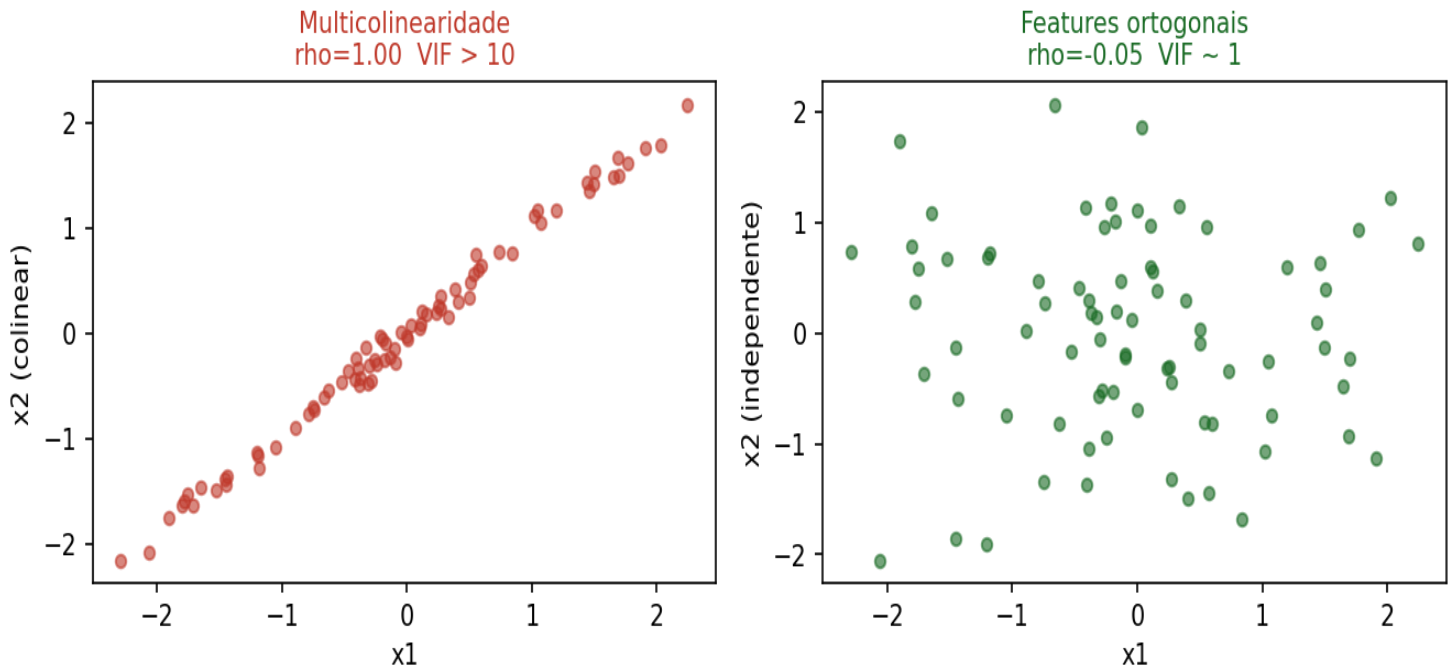

- **MI mede a redução de incerteza sobre Y ao observar X**
- Detecta qualquer tipo de dependência: linear, não-linear, discreta
- *MI = 0* => X e Y são independentes
- *MI > 0* => X carrega informação sobre Y
- **sklearn:**
  - *mutual_info_classif*
  - *mutual_info_regression*

# Métodos Filter

- **Exemplo:** Multicolinearidade e VIF (*Variance Inflation Factor*)

- **VIF (Variance Inflation Factor):** mede o quanto uma feature pode ser explicada pelas outras features do modelo

- **VIF = 1: sem correlação** com as outras features
- **VIF > 5:** correlação **preocupante** | **VIF > 10:** colinearidade **severa**




**Quando há colinearidade,** os coeficientes do modelo ficam instáveis

# Métodos Filter

- **Teste Qui-Quadrado ($chi^2$)**
  - Testa independência estatística entre feature e target
  - Adequado para features categóricas e targets de classificação
    - **H0:** feature e alvo são independentes
    - ***p*-valor baixo (<0.05):** feature é relevante
    - **sklearn:** *chi2 + SelectKBest*
  - **Limitação:**
    - Requer features não-negativas
    - Não captura interações

- **Teste ANOVA**
  - Testa se a media do target difere entre grupos definidos pela feature categórica
  - Adequado para features contínuas (média só faz sentido em variáveis contínuas)
    - **H0:** médias do target são iguais em todos os grupos
    - ***p*-valor baixo (<0.05):** feature é relevante
    - **sklearn:** *f_regression ou f_classif* + SelectKBest
  - **Limitação:**
    - Assume relação linear entre feature e target
    - Sensivel à outliers no target

# Métodos Filter

| Método | Feature | Target | Problema | Por que funciona |
|---|---|---|---|---|
| **chi²**<br>`chi2` | discreta | categórico | classificação | Conta frequências observadas vs esperadas por categoria.<br>Discreto → contagens; classificação → classes no target. |
| **f_classif**<br>`f_classif` | contínua | categórico | classificação | ANOVA: classes do target definem os grupos; testa se a média da feature contínua difere entre eles. |
| **f_regression**<br>`f_regression` | contínua | contínuo | regressão | Testa se a correlação linear feature→target é ≠ 0.<br>Sem grupos — regressão univariada; ambos os lados numéricos. |
| **Pearson / MI**<br>`mutual_info` | contínua | qualquer | qualquer | Contínua não forma grupos — chi² e ANOVA não se aplicam.<br>Pearson: linear. MI: qualquer tipo de dependência. |

# O que é um Método Wrapper?

- **Ideia central:**
  - Usar o próprio modelo preditivo como critério de qualidade de um subconjunto

- **Diferente de Filter:**
  - Avalia o subconjunto como um todo, capturando interações entre features

- **Fluxo:**
  - subconjunto → modelo → métrica → ajusta subconjunto → repete
  - A avaliação é sempre feita com validação cruzada para evitar overfitting na seleção

- **Custo:** para o treino com *p* features, pode exigir $O(p^2)$ a $O(2^p)$

- **Vantagem:** o subconjunto é **ótimo PARA aquele modelo específico**

# Forward Selection: Construindo do Zero

**Forward Selection — Parte do conjunto vazio e adiciona uma feature por iteracao**

**Inicio**
conjunto vazio { }

**Candidatas:**
todas as p features
selecionadas: nenhuma

*O(p) treinos na 1a iteracao*

**Testar candidatas**
adicionar cada uma temporariamente

Para cada feature candidata:
treina modelo com atual + ela
mede metrica (CV ou holdout)

*O(p - k) treinos por iteracao*

**Adicionar melhor**
a que mais melhora a metrica

Inclui a feature de maior
ganho na metrica
Remove-a das candidatas

*greedy: decisao definitiva*

**ganho > threshold?**

NAO

**PARAR**
conjunto atual = resultado final
executar .transform(X)

SIM

*volta ao passo Testar com conjunto expandido*

**Greedy**
Decisoes anteriores nao
sao revisitadas

**Vantagem:**
Rapido nas primeiras iteracoes
Eficiente quando subconjunto otimo e pequeno

**mlxtend:**

```
SequentialFeatureSelector(estimator, direction='forward')
```

**sklearn:**

```
SequentialFeatureSelector(direction='forward')
```

# Forward Selection — Evolução da Performance

- **Iteração 1:**
  - Feature mais poderosa sozinha → ex: já atinge 91% de acurácia
- **Iteração 2-5:** ganhos expressivos a cada adição (+2% a +3% por feature)
- **Iteração 6-10:** ganhos marginais decrescentes (+0.5% ou menos por feature)
- ***Plateau*:** 12+ features, adicionar não melhora (pode até piorar por overfitting)
- **Gráfico de curva:**
  - Eixo x = número de features selecionadas | Eixo y = acurácia CV
- **Conclusão visual:**
  - O 'joelho da curva' é o ponto ótimo: máximo ganho / mínimas features

# Backward Elimination: Removendo o Desnecessário

**Backward Elimination — Parte do conjunto completo e remove uma feature por iteracao**

**Inicio**
conjunto completo { p features }

**Custo inicial:**
treina com TODAS as p features — maior custo

*O(p) treinos na 1a iteracao*

**Testar remocoes**
remover cada uma temporariamente

Para cada feature do conjunto:
treina sem ela, mede metrica
verifica queda de performance

*O(k) treinos por iteracao*

**Remover pior**
que menos prejudica a metrica

Elimina a feature cuja remocao mais melhora (ou menos prejudica)

*greedy: decisao definitiva*

**remover causa queda signif.?**

SIM

**PARAR**
conjunto atual = resultado final
executar .transform(X)

NAO

*volta ao passo Testar com conjunto reduzido*

**Preferir ao Forward**
quando features tem forte interacao entre si

**vs Forward:**
Custo inicial maior (treina com todas)
Melhor quando muitas features sao relevantes

**mlxtend:**

```
SequentialFeatureSelector(estimator, direction='backward')
```

**sklearn:**

```
SequentialFeatureSelector(direction='backward')
```

# Forward vs Backward: Quando Cada Direção Faz Sentido

- **Forward é melhor quando:**
  - O subconjunto ótimo é pequeno (< 20% das features totais)
  - O dataset tem muitas features e treinar o modelo completo é caro
- **Backward é melhor quando:**
  - Features têm forte correlação e se complementam mutuamente
  - Você suspeita que quase todas as features são relevantes
- **Ambos são greedy:**
  - Podem ficar presos em ótimos locais → combine com **RFECV** para confirmar
- **Na prática:**
  - Forw. + Backw. com validação cruzada convergem para resultados similares

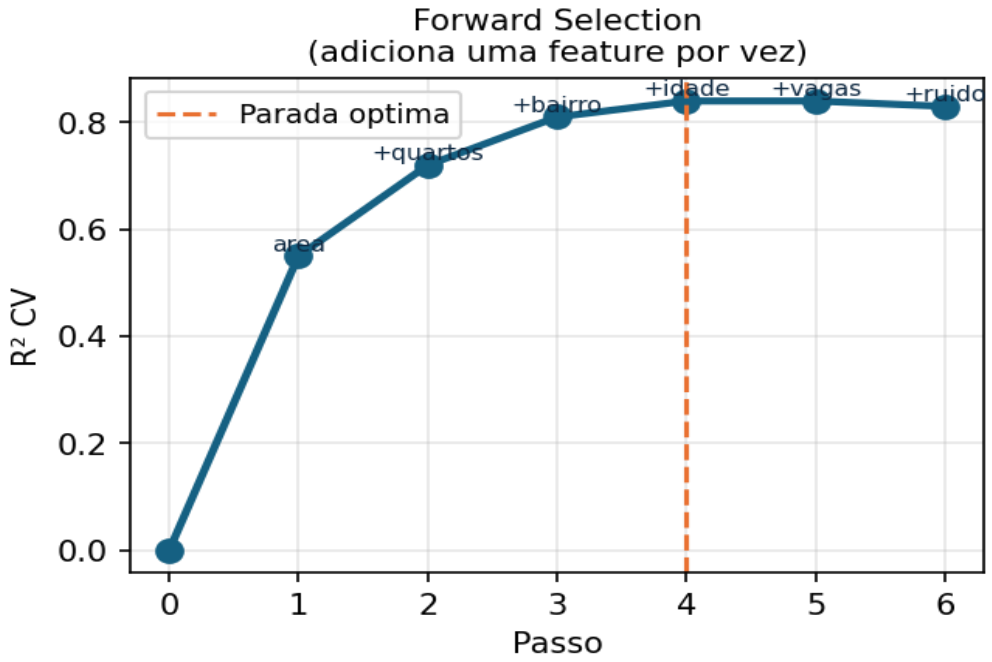
# Wrapper: Forward e Backward Selection

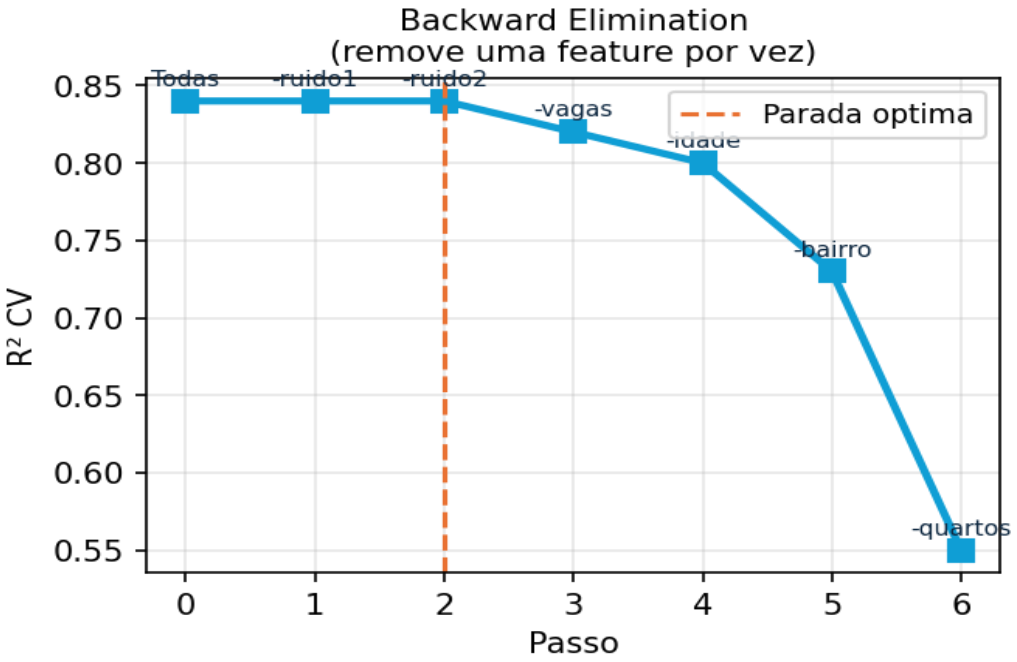
**Forward Selection:** Começa vazio, adiciona a melhor feature a cada passo

**Backward Elimination:** Começa com todas, remove a pior a cada passo

**Critério de parada:** *CV-score* não melhora além de *épsilon* (*threshold*)

**Custo:** $O(p^2)$ avaliações — viável para p < 50



# RFE: Ranquear, Eliminar e Repetir

## RFE: Recursive Feature Elimination

**RFE — Recursive Feature Elimination: 3 passos que se repetem ate restar n_features_to_select**

**Passo 1 — Treinar**
modelo com features atuais

Ajusta coeficientes ou importancias de cada feature

*qualquer estimador sklearn*

**Passo 2 — Ranquear**
features por importancia

Coeficientes (Lasso, Ridge)
MDI (Random Forest)

*rank 1 = mais importante*

**Passo 3 — Remover**
as N menos importantes

Remove step features do conjunto atual

*n_features_to_remove*

**Features**
= n_select?

**SIM**

**PARAR**
n_features_to_select mantidas
executar .transform(X)

**NAO**

*volta ao passo 1 com conjunto reduzido*

**Sklearn:**

```
RFE(estimator=LogisticRegression(), n_features_to_select=10, step=1)
```

*step=1: remove 1 feature por iteracao (step > 1 e mais rapido)*

# RFE — Como o Ranking de Importância Evolui

**Apos o RFE:**

`rfe.support_` → True = selecionada

`rfe.ranking_` → 1 = selecionada; >1 = iteracao em que foi removida

# O Problema do RFE Simples

- RFE exige que você **defina *n_features_to_select* ANTES de rodar**
- **Como saber qual é o número ótimo?** Na maioria dos casos, você não sabe
- Definir **arbitrariamente** pode **manter** features **inúteis** ou **remover** feats **valiosas**
- **Solução ingênua:**
  - testar vários valores de *n* → mas isso é custoso e propenso a *overfitting*
- **Pergunta certa:**
  - 'Qual é o menor conjunto que maximiza a performance com CV?'
- **Resposta:**
  - RFECV: deixa a validação cruzada responder essa pergunta

# RFECV: A Validação Cruzada Escolhe o Número Ideal

**RFECV — RFE + Grid Search automatico sobre k via cross-validation**

**Inicio**
conjunto completo { p features }
**k atual = p**
armazena score CV
para cada k de 1 a p
*p x k treinos no total*

**RFE: remove 1**
feature menos importante
Ranqueia por coeficiente
ou importancia
Elimina a de menor rank
*k = k - 1*

**Avaliar com CV**
k-fold no subconjunto atual
Treina e valida em k folds
Armazena score medio
para este tamanho de k
*cv_results_[k] = mean_score*

**k = 1 atingido?**

SIM → **Selecionar k***
menor k com score maximo
rfecv.n_features_ = k*

NAO → *volta: remove mais 1 e reavalia CV*

**Sklearn:**

```
RFECV(estimator=SVC(kernel='linear'),
cv=5, scoring='accuracy',
min_features_to_select=1)
```

**Resultados:**

`rfecv.n_features_` → numero otimo de features (k*)
`rfecv.cv_results_['mean_test_score']`
→ curva de performance por k (para plotar)

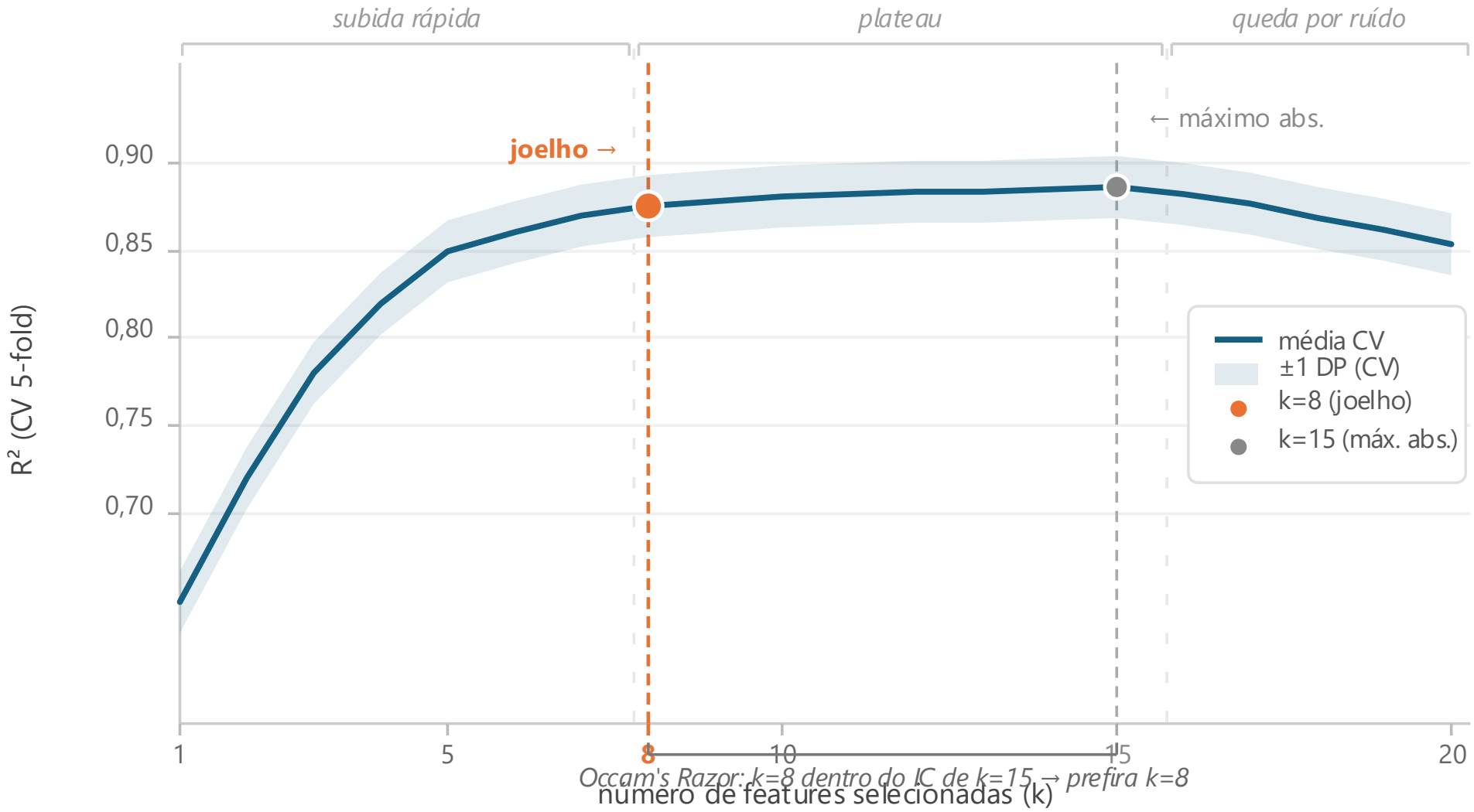
**Custo:**

**p × k treinos**
ex: p=30, k=5
**= 150 treinos**

# RFECV — Curva de Performance vs Número de Features

- **Eixos do gráfico:**
  - X: número de features utilizadas (1 até p)
  - Y: performance do CV ± desvio padrão (faixa sombreada)
- **Padrão típico:**
  - subida rápida com as primeiras features, *plateau*, possível queda por ruído
- **Ponto ótimo:**
  - ‘Joelho' antes do plateau, não necessariamente o máximo absoluto
- **Faixa sombreada importa:**
  - Se performance com 8 features está dentro do intervalo do máximo com 15, prefira 8
  - *Occam's Razor*: entre modelos igualmente bons, prefira o mais simples

# RFECV — Curva de Performance vs Número de Features



# Custo Computacional dos Wrappers

- **Forward Selection:**
  - $O(p^2/2)$ treinamentos no pior caso — $p=100$ → ~5.000 treinos
- **Backward Elimination:**
  - Similar ao Forward, mas o primeiro modelo tem *p* features
- **RFE com step=1:**
  - exatamente p treinamentos (um por iteração de remoção)
  - RFECV com step=1 e k=5 folds: p × k = p×5 treinamentos → p=100 → 500 treinos
- **Regra prática:**
  - Wrappers são viáveis com **p < 100** e **modelos simples** (*Logistic*, *LinearSVC*)
  - Com p > 200 ou modelos lentos (RF, XGBoost):
    - Usar Filter para pré-descarte antes do Wrapper

# Quando Wrappers se Tornam Inviáveis

- **Cenário 1:** $p > 500$ features e modelo não-linear → dias de processamento
- **Cenário 2:** dataset com milhões de amostras → cada treinamento já é lento por si só
- **Cenário 3:** produção com re-treino contínuo → custo de seleção não pode ser recorrente

- **Alternativa 1:** Filter ($chi^2$, ANOVA, correlação) para descarte rápido de 80% das features
- **Alternativa 2:** Lasso/Elastic Net como Embedded — seleção com custo de um único treinamento
- **Alternativa 3:** Feature Importance de Random Forest — rápida e eficaz para alta dimensionalidade

# O que são Métodos Embedded?

- A seleção de features ocorre como **parte intrínseca do processo** de treinamento
- **Não há loop externo de avaliação** — o modelo e a seleção aprendem juntos
- Mais **eficientes** que Wrappers: seleção com o custo de **um único treinamento**
- Mais **precisos** que Filters: o modelo **considera interações** entre features **naturalmente**
- **Exemplos canônicos:**
  - Lasso (L1), Ridge (L2), Elastic Net (L1+L2), Random Forest, Gradient Boosting
- **Na prática:**
  - Métodos mais usados em produção por seu equilíbrio custo-qualidade

*CONSULTAR: Exercício 04 do script de apoio*

# Regularização L1 (Lasso): A Penalização que Zera Coeficientes

- **Função objetivo:**

$$\min_{\beta} \frac{1}{2n} \|y - X\beta\|_2^2 + \alpha \|\beta\|_1$$

- **Propriedade única do L1:**
  - Empurra coeficientes exatamente para zero (esparsidade)

- **Features com coeficiente zero são automaticamente excluídas por seleção implícita**
  - **α pequeno** → poucos coeficientes zerados; modelo retém mais features; risco de overfitting
  - **α grande** → muitos coeficientes zerados; modelo mais esparso; risco de **underfitting** se α excessivo

- **Scikit:**
  - *Lasso(alpha=0.01) | LassoCV() para encontrar alpha ótimo via CV*

# Regularização L2 (Ridge): Encolher sem Zerar

- **Função objetivo:**
  - $$\mathrm{RSS}_{\mathrm{ridge}} = \sum_{i=1}^{n} \left[y_i - (\mathbf{w} \cdot \mathbf{x}_i + b)\right]^2 \boxed{+\alpha \sum_{j=1}^{p} w_j^2}$$

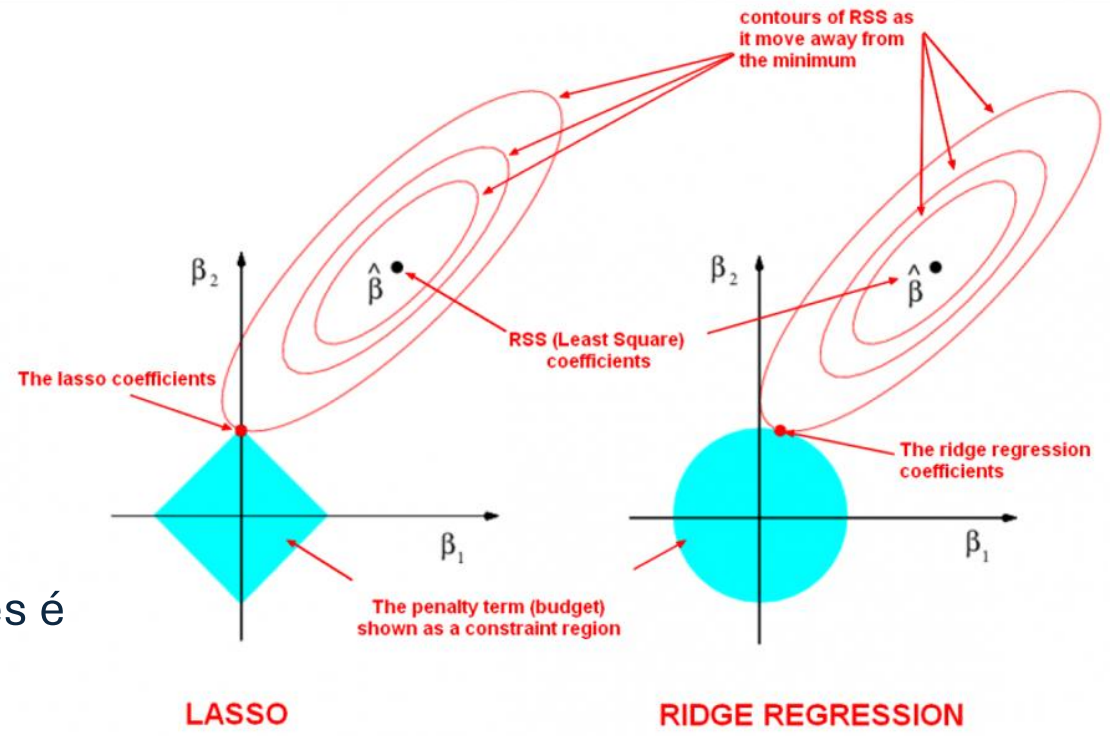
    regularização $\ell_2$

- **Ridge encolhe** coeficientes, mas **raramente** os **zera completamente**
- Com **multicolinearidade**: **distribui o peso** entre features correlacionadas
  - **Lasso:** tende a manter uma e zerar as outras **arbitrariamente**

- **Lasso** é preferível quando suspeita-se que **muitas features** são **irrelevantes** ou **redundantes**
- **Ridge** é preferível quando **todas as features contribuem** e a **interpretabilidade não é prioridade**

# Visualização Geométrica: Por que L1 Produz Esparsidade

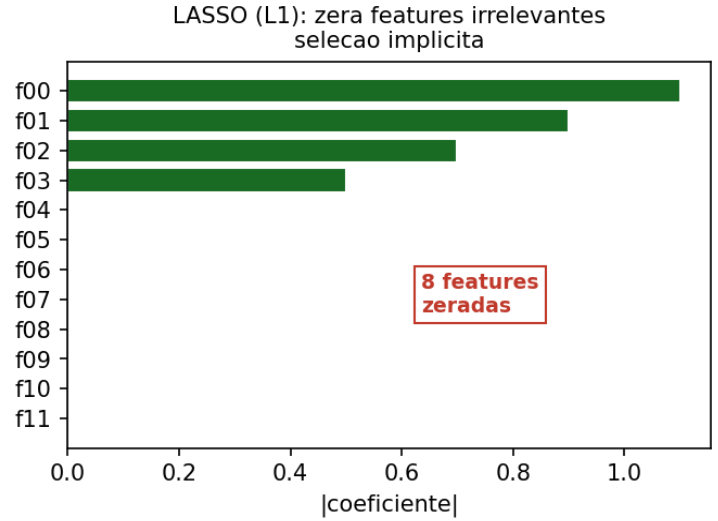
- **Geometria do L1:**
  - região de restrição é um losango (diamante em 2D)
- **Geometria do L2:**
  - região de restrição é uma esfera (círculo em 2D)
- **A solução ótima:**
  - Onde a elipse de RSS toca a região de restrição pela primeira vez
- **L1 (losango):**
  - As 'pontas' estão nos eixos onde um dos coeficientes é exatamente zero = feature excluída
- **L2 (esfera):**
  - Sem pontas angulosas → solução raramente está exatamente num eixo → não zera

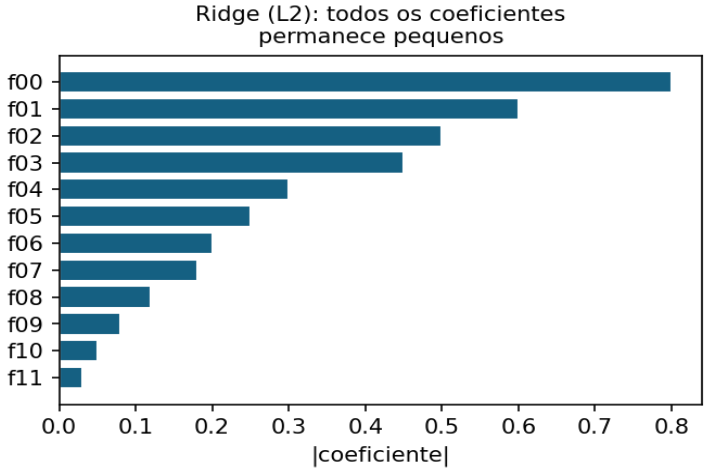



**Fonte Imagem:** Lasso vs Ridge Regularization

# Visualização Geométrica: Por que L1 Produz Esparsidade

- **Geometria do L1:**
  - região de restrição é um losango (diamante em 2D)
- **Geometria do L2:**
  - região de restrição é uma esfera (círculo em 2D)
- **A solução ótima:**
  - Onde a elipse de RSS toca a região de restrição pela primeira vez
- **L1 (losango):**
  - As 'pontas' estão nos eixos onde um dos coeficientes é exatamente zero = feature excluída
- **L2 (esfera):**
  - Sem pontas angulosas → solução raramente está exatamente num eixo → não zera



# Lasso vs Ridge: Quando Cada Um é a Escolha Correta

- **Use Lasso quando:**
  - O número real de features relevantes é pequeno (esparsidade esperada)
  - Interpretabilidade é crítica e você precisa justificar cada feature incluída
  - Features correlacionadas existem mas você quer apenas uma representante

- **Use Ridge quando:**
  - A maioria das features contribui marginalmente e você não quer descartá-las
  - Features são altamente correlacionadas e a estabilidade importa mais que esparsidade

- **Dúvida?**
  - Elastic Net combina os dois e tem desempenho robusto em quase todos os cenários

# Elastic Net: Combinando L1 e L2

- **Função objetivo combina Lasso e Ridge:**

$$\hat{\beta} \equiv \underset{\beta}{\operatorname{argmin}}(\|y - X\beta\|^2 + \lambda_2 \|\beta\|^2 + \lambda_1 \|\beta\|_1)$$

- **Parâmetro *l1_ratio*:**
  - Controla o mix $\boldsymbol{\lambda_1} / \boldsymbol{\lambda_2}$ — *α=1* é Lasso puro, *α=0* é Ridge puro

- **Parâmetro *alpha*:**
  - Controla a intensidade total da regularização

- Resolve limitação do Lasso com features correlacionadas: incluir ou excluir grupos

- **Na prática:**
  - *l1_ratio=0.5* é um ponto de partida sólido; otimize com *ElasticNetCV*()

- **Recomendado** como **padrão** quando você **não tem hipótese forte** sobre a **estrutura** do **problema**

# Elastic Net: O que Cada Parâmetro Controla

- ***alpha*:** força total da regularização
  - alpha=0: sem penalização (OLS)
  - alpha=1: regularização intensa
  - ajuste via CV

- ***l1_ratio*:** proporção L1 no mix
  - 0.0: puro Ridge (sem zeros)
  - 1.0: puro Lasso (muitos zeros)
  - 0.5: equilibrado

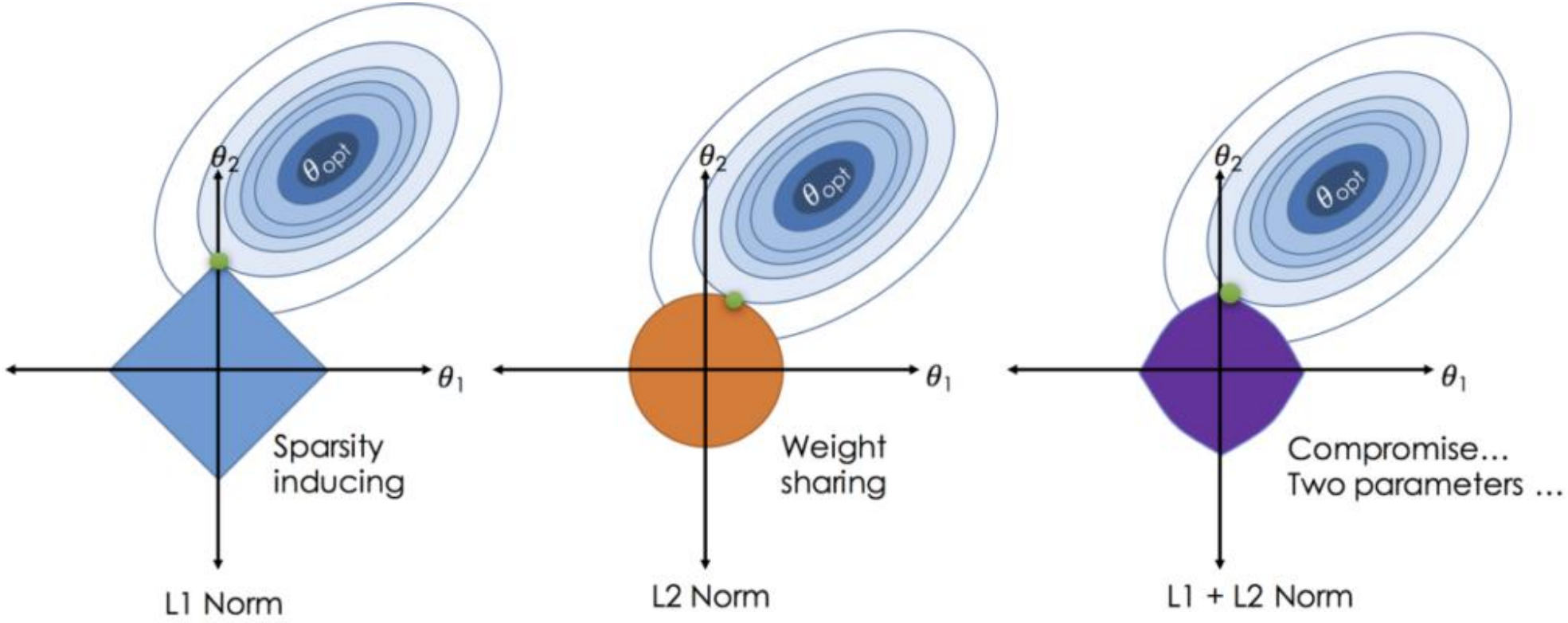
- **Interação:** alpha alto + l1_ratio alto:
  - máxima esparsidade (poucas features)

```
# Criar e treinar o modelo ElasticNetCV
# Busca automática de alpha e l1_ratio via validação cruzada
model = ElasticNetCV(
    l1_ratio=[0.1, 0.5, 0.9],       # proporção L1/L2 a testar
    alphas=np.logspace(0, 0.5, 1),  # valores de alpha a testar
    cv=5,                           # 5-fold cross-validation
    random_state=42
)

model.fit(X_train, y_train)

# Previsões
y_pred = model.predict(X_test)
```

- **Grid search recomendado:**
  - ElasticNetCV(l1_ratio=[0.1, 0.5, 0.7, 0.9, 1.0], cv=5)

# Elastic Net: Trajetória dos Coeficientes

- **l1_ratio=1 (Lasso) / l1_ratio=0 (Ridge) / l1_ratio=0.5:** comportamento intermediário — alguns zeros, outros apenas pequenos




**Fonte Imagem:** Machine Learning- ElasticNet Regression

- **Interpretação:** features que zeram com alpha baixo são menos relevantes

# SelectFromModel: O Conector com Pipelines

- **SelectFromModel:**
  - Transforma qualquer modelo com coeficientes ou importâncias em um seletor
  - Funciona com: Lasso, ElasticNet, RandomForest, XGBoost...
- **Parâmetro threshold:** define o corte de importância para incluir uma feature
  - ***'mean':*** features acima da importância média são mantidas
  - ***'median':*** metade das features é mantida
  - ***valor numérico:*** ex. threshold=0.01 mantém features com importância > 1%

- **Sklearn:** *SelectFromModel(estimator=Lasso(alpha=0.01), threshold='mean')*

# SelectFromModel dentro de um Pipeline

- **Integração natural com Pipeline do sklearn:** evita data leakage automaticamente

```
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('selector', SelectFromModel(Lasso(alpha=0.005))),
    ('model', LogisticRegression())
])
```

- O *scaler* e o *seletor* são ajustados SOMENTE nos dados de treino de cada fold
- **Vantagem:** a seleção de features é parte do modelo e não uma etapa separada e esquecível

# Threshold no SelectFromModel

- **Escolhendo o Corte:**
  - ***threshold='mean'*** → mantém features com importância > média: bom ponto de partida
  - ***threshold='median'*** → descarta metade das features: mais agressivo, bom para alta dimensionalidade
  - ***threshold=0.0*** → mantém apenas features com coeficiente não-zero (ideal após Lasso)
  - ***threshold='1.25*mean'*** → mais conservador: mantém apenas features claramente acima da média

- **Dica:** plotar a distribuição de importâncias antes de definir o threshold
- **Para o Lasso:** *threshold=1E-5* é suficiente para separar zeros dos não-zeros

# Feature Importance em Árvores: O que *feature_importances_* Mede

- **Em árvores de decisão:**
  - Mede a redução média de impureza (Gini ou entropia) causada pela feature
  - Uma feature usada em splits altos da árvore e com grande redução de impureza recebe score alto

- **Em Random Forest:**
  - Média das importâncias de todas as árvores do ensemble
  - **Propriedade:** importâncias somam 1.0 — são proporções **relativas**, não magnitudes absolutas

- **Sklearn**: *rf.feature_importances_* retorna array com uma importância por feature
- **Limitação:** o método favorece artificialmente features com alta cardinalidade

# O Viés do *feature_importances*: Alta Cardinalidade

- Features com muitos valores únicos (ex: ID, ZIP code) criam mais splits possíveis
  - Mais splits → mais chances de reduzir impureza por acaso → score inflado artificialmente

- **Exemplo:**
  - Feature **aleatória** com 1000 **valores únicos** pode rankear acima de features **preditivas**
  - **Consequência:**
    - o ranking de importância nativa de RF não é confiável sem checagem
  - **Diagnóstico:**
    - se features contínuas de alta cardinalidade estão sempre no topo, investigue
  - **Solução:**
    - use Permutation Importance como verificação — ou como método principal

# Permutation Importance: A Abordagem Agnóstica

*CONSULTAR: Exercício 05 do script de apoio*

- **Ideia:**
  - Embaralhe aleatoriamente os valores de uma feature e meça quanto a performance cai
- **Se embaralhar causar grande queda:**
  - Ela é importante (o modelo dependia dela)
- **Se embaralhar não muda nada:**
  - a feature é irrelevante para aquele modelo
- **Agnóstico ao modelo:**
  - Funciona com qualquer estimador — não precisa de coeficientes ou árvores
- **Não tem viés de cardinalidade:**
  - embaralhar um ID ou um binário tem o mesmo custo
- **Sklearn:**
  - *sklearn.inspection.permutation_importance(model, X_val, y_val, n_repeats=30)*

# *feature_importances_* vs Permutation Importance: Comparação

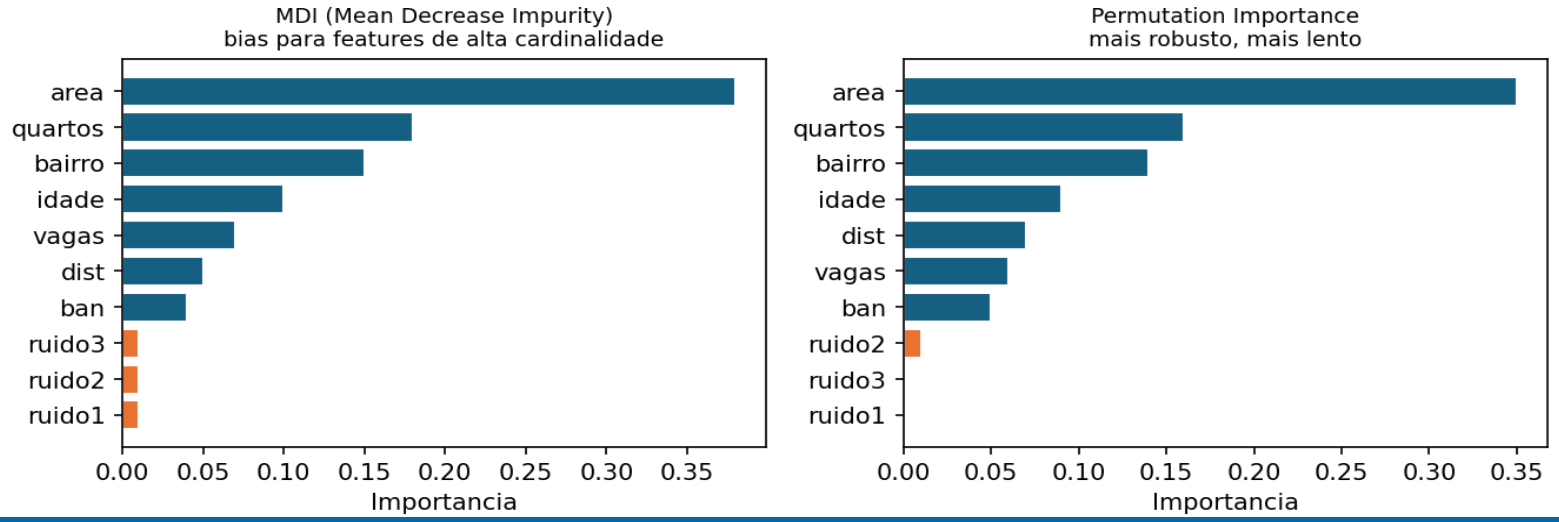
- ***feature_importances_* (Mean Decrease Impurity):**
  - Calculado durante o treino, rápido, baseado em impureza
  - Pode inflar features de alta cardinalidade
  - Medido nos dados de **TREINO**

- ***Permutation Importance*:**
  - Calculado pós-treino, mais lento, baseado na performance real
  - Sem viés de cardinalidade
  - Medido em dados de **VALIDAÇÃO** (mais honesto)

- **Quando concordam:** Sinal forte de que o ranking é confiável
- **Quando divergem:** Confie na Permutation Importance (referência mais robusta)
- **Recomendação:**
  - Usar MDI para exploração rápida e Permutation como validação final

# *feature_importances_* vs Permutation Importance: Comparação

- **MDI (*Mean Decrease Impurity*):**
  - Soma a redução de impureza de cada feature
  - Rápido mas tem bias para features de alta cardinalidade ou variância
- ***Permutation Importance*:**
  - Embaralha cada feature e mede queda no score
  - Mais robusto que MDI — detecta leakage e features espúrias



# Permutation Importance — Interpretando os Resultados

- **Resultado:**
  - Média e desvio padrão da queda de performance sobre *n_repeats* embaralhamentos

- **Importância positiva:**
  - Embaralhar prejudica → **feature é usada corretamente** pelo modelo
- **Importância próxima de zero:**
  - **Feature não importa** para este modelo nesta configuração
- **Importância negativa:**
  - Embaralhar melhora a performance → **feature pode estar introduzindo ruído!**
- **Intervalo de confiança largo:**
  - Feature com efeito **instável** → **pode não ser confiável em produção!**

- **Nunca avaliar Permutation Importance nos dados de treino:**
  - Isso mede memorização, não aprendizado

# Feature Importance: Random Forest vs XGBoost

- **Random Forest (MDI):**
  - Média de redução de impureza por feature sobre todas as árvores

- **XGBoost weight:**
  - Número de vezes que a feature foi usada em splits → favorece splits frequentes
- **XGBoost gain:**
  - Média do ganho de cada split que usa a feature → mais informativo que weight
- **XGBoost cover:**
  - Número de amostras cobertas pelos splits que usam a feature
- **Recomendação para XGBoost:**
  - Use *importance_type='gain'* — é o mais próximo da importância real

- **Melhor prática:**
  - **Combine gain (XGBoost) + Perm. Importance para um ranking robusto**

# O Problema da Instabilidade na Seleção de Features

*CONSULTAR: Exercício 06 do script de apoio*

- **Teste simples:**
  - Rode **RFECV** duas vezes com *random_state=42* e *random_state=99*
- **Resultado comum:**
  - Subconjuntos completamente diferentes com performance quase idêntica
- **Por quê?**
  - Múltiplos subconjuntos de tamanho similar frequentemente têm performance equivalente

- **Em produção:**
  - Instabilidade significa que o modelo muda com retreino sem motivo de negócio
- **Diagnóstico:**
  - *Bootstrap* o dataset 100 vezes e conte quantas vezes cada feature é selecionada
- **Solução:**
  - Prefira features selecionadas em > 80% dos *bootstraps* (realmente estáveis)

# Stability Selection: Selecionando com Confiança

**Stability Selection — Selecionando com Confianca via Bootstrap**

**Inicio**
dataset completo

Parametros:
N iteracoes de bootstrap
threshold π (ex: 0.8)

*i = 0 ; pi(j) = 0 para todo j*

**Bootstrap**
amostrar sem reposicao

Seleciona 50% das linhas
do dataset aleatoriamente
Subsample diferente em
cada iteracao

*i = i + 1*

**Selecao no subsample**
Lasso / RFE / qualquer metodo

Roda metodo de selecao
no subsample
Registra quais features
foram selecionadas

*selected[i][j] ← 1 ou 0*

**i = N iteracoes?**

SIM

NAO

*nova amostragem bootstrap*

**Calcular π(j) e declarar**
frequencia de selecao de cada feature

π(j) = #{selecoes de j} / N

**Estaveis:** π(j) > threshold
Nao estaveis: π(j) ≤ threshold

*bonus: controla falsos positivos*

**Implementacao:**
`RandomizedLasso` (sklearn, legado)
ou implementacao manual:
`Lasso + bootstrap loop`

**Resultado:**
Features com `π(j) > 0.8` = estaveis e confiaveis
Lista consistente: independe de um split especifico
*threshold tipico: 0.6 a 0.9 dependendo do rigor exigido*

**Custo:**
**N x custo(selecao)**
ex: N=100 bootstraps
100x mais lento que selecao simples

*CONSULTAR: Exercício 07 do script de apoio*

# Multicolinearidade e Seleção: O que Acontece com Features Correlacionadas

*CONSULTAR: Exercício 08 do script de apoio*

- **Lasso com features correlacionadas:**
  - Tende a escolher uma e zerar as outras arbitrariamente
  - O 'vencedor' pode mudar com pequeníssimas mudanças nos dados
  - **Isso explica parte da instabilidade:**
    - Duas features quase idênticas competem pelo mesmo papel

- **Elastic Net resolve parcialmente:**
  - Sua componente Ridge tende a incluir grupos de correlacionadas
- **VIF (*Variance Inflation Factor*):**
  - Detecta multicolinearidade antes da seleção
- **Boa prática:**
  - Remover features com VIF > 10 antes do Lasso → resultados mais estáveis

# Data Leakage na Seleção de Features

*CONSULTAR: Exercício 09 do script de apoio*

- **Leakage clássico:**
  - Selecionar features usando TODOS os dados antes de fazer o split treino/teste
- **Consequência:**
  - Dados de teste 'vazam' para a seleção → métricas infladas, modelo enganoso
- **Exemplo:**
  - SelectKBest no dataset completo → split → treinar → testar → resultado otimista

- **Regra absoluta:** qualquer transformação que usa o target deve ficar DENTRO do CV
- **Solução sklearn:**
  - Encapsular seleção + modelo em Pipeline → o CV ajusta tudo no *fold* de treino

- **Teste de sanidade:**
  - Comparar performance com e sem leakage (diferenças>5% = problema)

# Seleção de Features DENTRO do CV: O Jeito Correto

- **Errado:**
  - *selector.fit(X_all) → cross_val_score(model, X_selected, y)*
- **Certo:**
  - *Pipeline([('selector', selector), ('model', model)]) → cross_val_score(pipe, X_all, y)*
- **No Pipeline, para cada fold:**
  - *fit(X_treino) → transform(X_treino) → fit(model) → transform(X_val) → score*

- Dados de validação **NUNCA** influenciam o *selector.fit(), é* matematicamente correto
- **Atenção extra com RFECV:**
  - seu CV interno não pode se sobrepor ao CV externo de avaliação
- **Nested CV:**
  - CV externo para avaliação | CV interno para seleção
  - Mais correto, mais caro

# Wrappers vs Embedded: Comparativo Completo

| Dimensão | Wrappers (RFE, RFECV) | Embedded (Lasso, RF) | Vantagem |
|---|---|---|---|
| **Custo computacional** | $O(p^2)$ treinamentos por fold<br>*cresce muito com p grande* | O(1) treinamento<br>*seleção embutida no ajuste* | **Embedded** |
| **Qualidade do resultado** | Ligeiramente superior<br>*modela interações entre features* | Boa, mas não avalia subconjuntos<br>*cada feature avaliada isoladamente* | **Wrappers** |
| **Estabilidade** | Sensíveis ao split de treino<br>*subconjunto pode mudar entre runs* | Mais estáveis<br>*Lasso com L1 fraco é consistente* | **Embedded** |
| **Interpreta-bilidade** | Ranking de subconjuntos<br>*"features 1, 3 e 7 foram escolhidas"* | Coeficientes diretos (Lasso/EN)<br>*$\beta=0 \rightarrow$ descartada; $\beta\neq0 \rightarrow$ magnitude importa* | **Embedded** |
| **Flexibilidade de modelo** | Funciona com qualquer estimador<br>*SVM, KNN, regressão — qualquer um* | Dependem do modelo base<br>*Lasso só para linear; RF para árvores* | **Wrappers** |
| **Recomendação** | Use RFECV para validar e refinar<br>*quando $p < 50$ e precisão é crítica* | Comece sempre aqui<br>*eficiente; use LassoCV ou RF importance* | **Embedded → RFECV** |

# Pipeline Completo com RFECV + Modelo Final

**Pipeline(steps=[scaler, rfecv, svc])**

**Etapa 1**
Pré-processamento
`StandardScaler()`
*necessário para SVC e Lasso*

**Etapa 2**
Seleção — RFECV
`estimator=LinearSVC(C=1)`
`step=1 · cv=5 · accuracy`
*escolhe k ótimo via CV*

**Etapa 3**
Modelo final
`SVC(kernel='rbf', C=10)`
*só com features selecionadas*

**Avaliação**
cross_val_score
`cv=StratifiedKFold(5)`
`scoring='accuracy'`
*dentro de 0,5% do baseline*

**Em produção:**
`pipe.fit(X_train, y_train) → joblib.dump(pipe, 'model.pkl')`

*CONSULTAR: Exercício 10 do script de apoio*

# Pipeline Completo com SelectFromModel + Elastic Net

**Pipeline(steps=[scaler, selector, logreg])**

**Etapa 1**
Pré-processamento

`StandardScaler()`

*normaliza para L1+L2*

**Etapa 2**
Seleção — SelectFromModel

`ElasticNetCV(l1_ratio=[.5,.9,1.])`

`cv=5 · threshold=1e-5`

*alpha e l1_ratio ótimos via CV*
*mantém coef. não-nulo (>1e-5)*

**Etapa 3**
Modelo final

`LogisticRegression(C=1.0)`

*features sobreviventes*

**Vantagens**

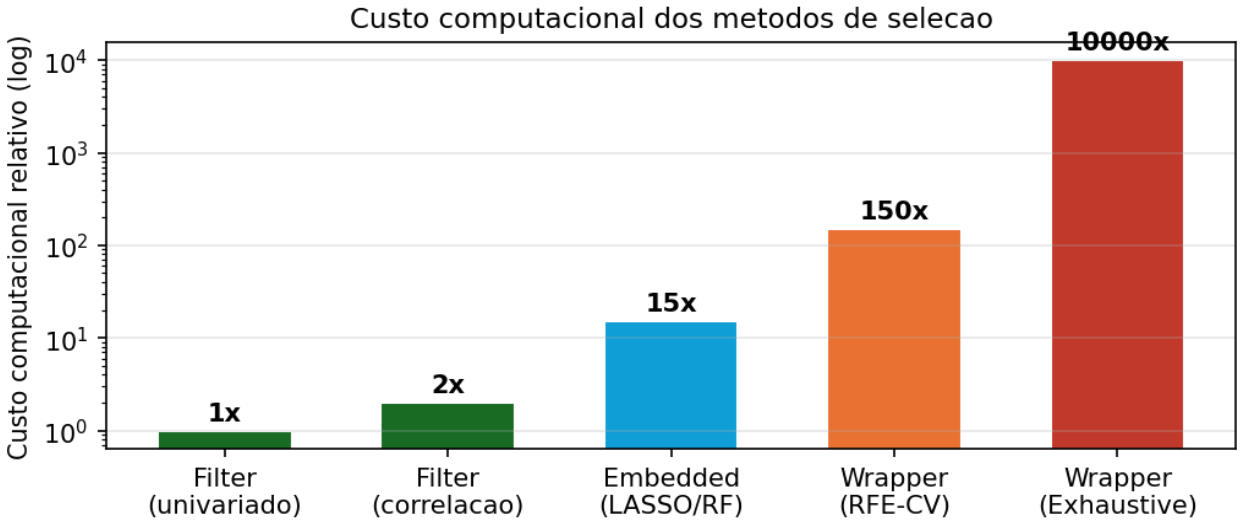
- Interpretável (coef. logísticos)
- Rápido e robusto a leakage
- Quando: linear/quasi-linear, p<500

**Em produção:**

```
pipe.fit(X_train, y_train) → joblib.dump(pipe, 'model.pkl')
```

# Como Escolher o Método Certo

- **Fatores a serem considerados:**
  - Tamanho do dataset
  - Tipo de modelo
  - Estabilidade e Interpretabilidade
  - Recursos computacionais




- **Dataset:** > 10k linhas e > 50 features → evite Wrappers (custo $O(p^2)$)
- **Modelo:** linear → Lasso/Elastic Net | Árvores → Feature Importance nativa
- **Estabilidade:** Seleção instável em produção → Stability Selection ou Embedded
- **Interpretabilidade:** Prefira Embedded com coeficientes explícitos
- **Recursos computacionais limitados:** Filter ou Embedded sempre antes de Wrapper
- **Regra prática:** Comece com Filter para descarte rápido, refine com Embedded/RFECV

# Como Reportar Resultados de Seleção

## Adapte ao público

### Time técnico

**Curva RFECV: N features × acurácia CV**
*+ intervalo de confiança*

**Tabela comparativa dos métodos**
*N features, performance, tempo*

### Stakeholders

**Mensagem em linguagem simples**

*"Reduzimos de 30 para 13 features mantendo 99,8% da acurácia"*

Sem jargão técnico

### Negócio

**Liste as features selecionadas com interpretação de domínio**

*Ex: 'Idade do cliente'*
*Evite: nomes técnicos ('feature_07')*

## Sempre inclua

### Documente — auditável

Quais features foram descartadas
Por qual método (Lasso, RFECV, VIF, chi$^2$)
*Garante rastreabilidade, reprodutibilidade e auditabilidade*

### Estabilidade por bootstrap

% de vezes que cada feature foi selecionada
Via bootstrap com N repetições do dataset
*Features com % baixa = sinal fraco ou instável*

# Seleção em Datasets Grandes: Estratégias Práticas

**Pipeline de 5 fases — viável mesmo com p > 1 000 features**

**Fase 1 — Filter**

**Descarte rápido**

`corr. target < 0.01`

*remove não-correlacionadas*

**Fase 2 — Redundância**

**Drop colineares**

`corr. mútua > 0.95`

*remove redundantes*

**Fase 3 — Embedded**

**Seleção implícita**

`Lasso / ElasticNet`

`GridSearchCV (α)`

**Fase 4 — Validação**

**Confirm. de relevância**

`Permutation Importance`

*nas features sobreviventes*

**Fase 5** (opcional)

**Refinamento final**

`RFECV (~30-50 features)`

*maior precisão, maior custo*

**Fases 1–2: velocidade | Fase 3: seleção | Fase 4: validação | Fase 5: refinamento (se p < 50 restantes)**

*Resultado esperado: de p > 1 000 features para ~10-30 features com mínima perda de performance*

# Guia de Decisão: Fluxograma Prático

**① p < 50 features**
modelo simples?

→ **RFECV + LinearSVC / Logistic**

*máxima precisão para datasets pequenos*

**② p entre 50 e 200**
suspeita esparsidade?

→ **Lasso ou Elastic Net**

*esparsidade + seleção implícita*

**③ p entre 50 e 200**
features correlacionadas?

→ **Elastic Net (l1_ratio=0.5)**

*balanço L1+L2 para colineares*

**④ p > 200**
ou modelo lento?

→ **Filter → Embedded**

*pré-descarte + seleção implícita*

**⑤ Modelo é árvore**
ou ensemble?

→ **Permutation Importance**

*+ SelectFromModel (nativo para árvores)*

**⑥ Estabilidade crítica**
em produção?

→ **Stability Selection**

*aplica sobre qualquer método base*

**⑦ SEMPRE: encapsule em Pipeline — garante ausência de data leakage**

Encapsule qualquer seletor em Pipeline antes do cross_val_score — sem isso, o CV é inválido

# Implementação em *sklearn*

| Categoria | Classe / Função (sklearn) | Quando usar |
|---|---|---|
| **FILTER** | `VarianceThreshold(threshold=0.01)` | features quase constantes (variância ≈ 0) |
| | `SelectKBest(score_func=f_regression, k=10)` | regressão + feature contínua (correlação linear) |
| | `SelectKBest(score_func=mutual_info_regression)` | qualquer tipo de relação com target (linear ou não) |
| | `SelectKBest(score_func=chi2)` | classificação + feature discreta e não-negativa |
| **WRAPPER** | `RFE(estimator=Ridge(), n_features=10)` | p < 50, k fixo, funciona com qualquer estimador |
| | `RFECV(estimator=Ridge(), cv=5)` | p < 50, k ótimo escolhido automaticamente via CV |
| | `SequentialFeatureSelector(Ridge())` | forward / backward selection explícita |
| **EMBEDDED** | `Lasso(alpha=0.01) / LassoCV()` | regressão, suspeita de esparsidade |
| | `ElasticNet() / ElasticNetCV()` | regressão, features correlacionadas + esparsas |
| | `SelectFromModel(Lasso())` | seleção por coeficiente Lasso não-nulo |
| | `SelectFromModel(RandomForest())` | seleção por importância MDI (modelos em árvore) |
| | `permutation_importance(rf, X_te, y_te)` | importância pós-treino, agnóstica ao modelo |
| **PIPELINE** | `Pipeline([`<br>`('selector', SelectKBest(k=10)),`<br>`('scaler', StandardScaler()),`<br>`('model', Ridge()) ])` | encapsula seletor + modelo para CV correto<br>*sel.fit() só vê X_train do fold — sem data leakage* |

# Tabela Resumo: Pontos Fortes, Fracos e Quando Usar

| Método | Pontos Fortes | Pontos Fracos | Quando Usar |
|---|---|---|---|
| WRAPPER<br>**Forward/Backward** | Intuitivo, good accuracy | $O(p^2)$, greedy | **p < 50, iterações rápidas** |
| **RFE** | Ranking explícito | Requer N pré-definido | **Quando N é conhecido** |
| **RFECV** | N automático via CV | Mais lento que RFE | **Melhor wrapper geral** |
| EMBEDDED<br>**Lasso** | Rápido, interpretável | Instável c/ correlação | **Esparsidade esperada** |
| **Elastic Net** | Robusto, balanceado | 2 hiperparâmetros | **Padrão recomendado** |
| **Perm. Importance** | Agnóstico, sem viés | Lento (N permutações) | **Validação do ranking** |
| `feat_importances` | Embutido em árvores | Viés de cardinalidade | **Exploração rápida** |

# As 5 Armadilhas Mais Comuns em Produção

### ① Data leakage

Seleção fora do Pipeline

**O que acontece:**

Metricas infladas no CV

Modelo falha em producao

**Solucao:** encapsule em Pipeline()

```
Pipeline([selector, model])
```

### ② Ignorar estabilidade

Feature selecionada hoje pode sair amanha

**O que acontece:**

Subconjunto muda a cada re-treino

Pipeline de producao perde consistencia

**Solucao:** Stability Selection com bootstrap

*% selecao por feature em N repeticoes*

### ③ feat_importances_ sem Permutation

Rankings enganosos (alta cardinalidade)

**O que acontece:**

Features de alta cardinalidade inflariam

MDI — features reais ficam ocultas

**Solucao:** confirmar com Permutation Importance

*embaralha feature e mede queda no score*

### ④ Fixar N em RFE sem CV

Escolha arbitraria de numero de features

**O que acontece:**

N pequeno demais exclui sinal real

N grande demais inclui ruido e diminui generalizacao

**Solucao:** usar RFECV — N otimo via cross-validation

```
RFECV(estimator=Ridge(), cv=5)
```

### ⑤ Selecionar e 'esquecer'

Features relevantes mudam com data drift

**O que acontece:**

Distribuicao dos dados muda ao longo do tempo

Features selecionadas meses atras podem ser irrelevantes hoje

**Solucao:** re-selecionar periodicamente (ex: trimestral)

*monitorar data drift com testes de KS ou PSI*

**Regra de ouro:**

Trate a selecao de features como um componente do modelo, nao como um pre-processamento pontual

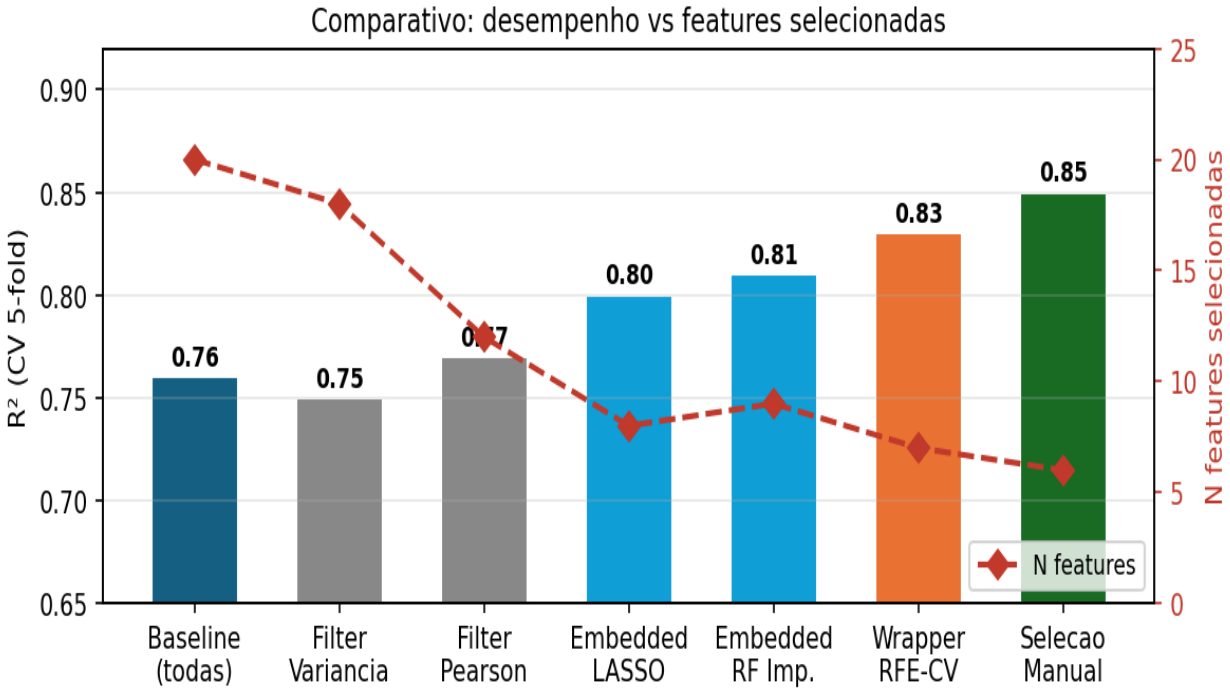
# Resumo da Aula e Próximos Passos

- Wrappers (RFECV): máxima qualidade para p moderado — use como referência de benchmark
- Embedded (Elastic Net + SelectFromModel): melhor equilíbrio para a maioria dos problemas reais
- Permutation Importance: validação obrigatória de qualquer ranking — use sempre em produção
- Pipeline é inegociável: qualquer seleção fora do Pipeline é potencialmente um erro de leakage
- Próximos passos recomendados:
- → Estude SHAP values para interpretabilidade pós-seleção
- → Explore Featuretools para seleção automatizada em dados relacionais
- → Aprofunde-se em Feature Stores (Feast, Tecton) para gestão em produção

# Resumo da Aula e Próximos Passos

- **Filter:**
  - Rápido, independente do modelo,
  - Não captura interações
- **Wrapper:**
  - Preciso, custoso, risco de
  - Overfitting na seleção
- **Embedded:**
  - Custo moderado, um único treino,
  - LASSO para esparsidade, RF para importância

- **REGRAS DE OURO:**
  - **Seleção dentro do Pipeline — sempre**
  - **Validar com CV — sempre**
  - **Baseline primeiro — sempre**
  - **Documentar decisões — sempre**



# O Dataset Utilizado no Script de Apoio: Breast Cancer Wisconsin

| 569 | 30 | 2 | 0 | 37% M / 63% B |
|---|---|---|---|---|
| amostras | features contínuas | classes (M / B) | valores ausentes | dataset desbalanceado |

**Origem:**
Wolberg, Mangasarian, Street — Univ. Wisconsin, 1993
Imágens de biópsia por agulha fina (FNA) de massa mamária
Medidas nos núcleos celulares das imagens digitalizadas
*DOI: 10.24432/C5DW2B — UCI ML Repository #17*

**Estrutura das 30 features:**
10 propriedades medidas em 3 estatísticas cada:
- **mean_*** — média entre todos os núcleos da imagem
- ***_error** — erro padrão (dispersão entre núcleos)
- **worst_*** — média dos 3 maiores valores

Total: 10 × 3 = **30 features contínuas**

**Tarefa:**
Classificação binária: Maligno (M = 212) vs Benigno (B = 357)
Baseline: ~97% acurácia com SVM linear (autores originais)
*sklearn: from sklearn.datasets import load_breast_cancer*

**As 10 propriedades do núcleo celular:**
- **radius** — distância média do centro até a borda
- **texture** — desvio padrão dos valores de cinza
- **perimeter** — perímetro do núcleo
- **area** — área do núcleo
- **smoothness** — variação local no comprimento do raio
- **compactness** — perímetro$^2$ / área − 1
- **concavity** — severidade das porções côncavas do contorno
- **concave points** — número de porções côncavas
- **symmetry** — simetria do núcleo
- **fractal dimension** — aproximação da linha costeira − 1

**Racional da utilização na aula:**
- **Multicolinearidade:** radius, perimeter e area são altamente correlacionados
- **Tamanho:** grande o suficiente para seleção, pequeno para visualizar
- **Interpretável:** coeficientes maiores = tumor maior / mais irregular
- **Impacto:** erros de classificação têm consequências médicas concretas

***Prof. Dr. Vinícius Horn Cene***

Departamento de Engenharia Elétrica

*DELET – UFRGS*

**Prédio do Instituto Eletrotécnico**

**Sala #206-D**

**e-mail:** vinicius.cene@gmail.com

**Linked-In:** https://www.linkedin.com/in/vinicius-cene-17695225/

**Research Gate:** https://www.researchgate.net/profile/Vinicius_Cene